Dynamic Programming

Coin Change

- Given an integer array of coins[ ] of size N representing different types of currency and an integer sum, The task is to find the number of ways to make sum by using different combinations from coins[].

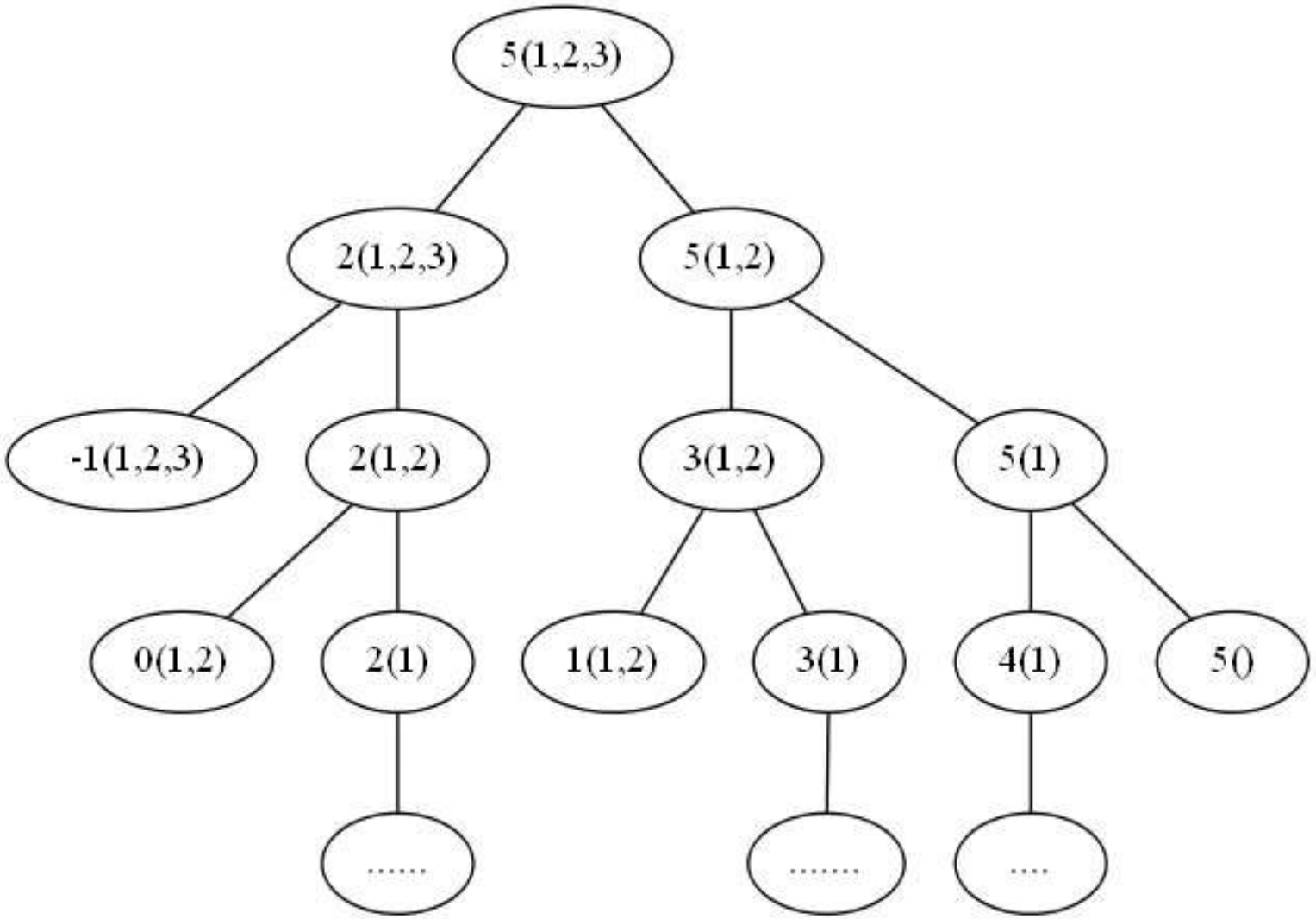
- Note: Assume that you have an infinite supply of each type of coin, and you are allowed to use same valued multiple times.

Coin Change

- ***Input:*** *sum = 4, coins[] = {1,2,3}, n =3*
  Output: *4*
  Explanation: *there are four solutions:*
    - *{1, 1, 1, 1}, {1, 1, 2}, {2, 2}, {1, 3}.*

- ***Input:*** *sum = 10, coins[] = {2, 5, 3, 6}, n =4*
  Output: *5*
  Explanation: *There are five solutions:*
    - *{2,2,2,2,2}, {2,2,3,3}, {2,2,6}, {2,3,5} and {5,5}.*

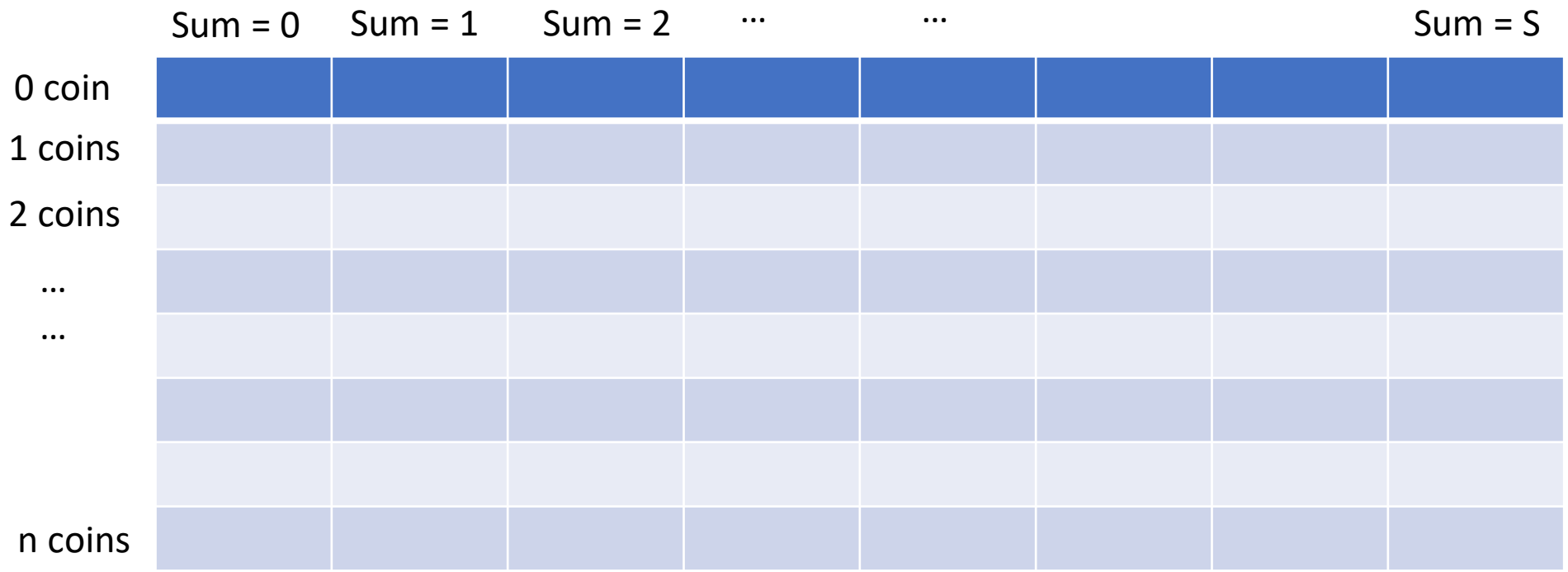
Coin Change



Coin Change

- Smaller sub problems:
  - change(sum,n) = change(sum, n-1) + change(sum - coins[n],n)

Coin Change Bottom-Up (Tabulation)



Coin Change Bottom-Up (Tabulation)

	Sum = 0	Sum = 1	Sum = 2	…	…			Sum = S
0 coin	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
1 coins	1							
2 coins	1							
…	1							
…	1							
	1							
	1							
n coins	1							

Bottom up implementation 1

```
for (i = 0; i <= n; i++)
        dp[i][0] = 1;
for (j = 1; i <= sum; i++)
        dp[0][j] = 0;
for (i = 1; i <= n ; i++) {
        for (j = 1; j <= sum; j++) {
                if(j < coins[i - 1])  //0 indexed coins array
                                dp[i][j] =  dp[i-1][j];
                else
                                dp[i][j] = dp[i-1][j] + dp[i] [ j – coins[i - 1] ];
        }
    }
    return dp[n][sum];
```

Coin Change Bottom-Up (2)

	Sum = 0	Sum = 1	Sum = 2	…	…			Sum = S
0 coin	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
1 coins	0	0	0	0	0	0	0	0
2 coins	0	0	0	0	0	0	0	0
…	0	0	0	0	0	0	0	0
…	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0
n coins	0	0	0	0	0	0	0	0

Bottom up implementation (2)

```
dp[0][0] = 1;

for (i = 1; i <= n ; i++) {
        for (j = 0; j <= sum; j++) {
            if(j < coins[i - 1])  //0 indexed coins array
                        dp[i][j] =  dp[i-1][j];
            else
                        dp[i][j] = dp[i-1][j] + dp[i] [ j – coins[i - 1] ];
        }
    }
    return dp[n][sum];
```

Top down solution?

Top down solution (Recursive)

- Initialize the dp array as -1 //to check whether the value is updated or not
- f(n, sum)
- Base Cases:
  - if(sum == 0) return dp[sum][n] = 1;
  - else if(n == 0) return dp[sum][n] = 0;
  - else if(dp[n][sum] != -1) return dp[n][sum];
  - else recursive call

Coin Change (Variations)

- Can only use each coins only once
- Each coins has limited supply (Can be different for each coin type)
- Only allowed to use k coins (all together)

Coin Change (Minimum Number of coins)

- Your goal is to find the minimum number of coins required, to give an exchange of a target (sum) by using n coins having values:
  - coins[] = {c1, c2, c3….cn}

Recurrence Relation

- Smaller sub problems and the relation between them ?

Recurrence Relation

< Excluding n^{th} coins and including n^{th} coins>

- Eexcluding:
  - dp[n-1][sum]
- Including:
  - 1 + dp[n] [sum – coins[n]]

Relation between them?

Coin Change (Minimum Number of coins)

< Excluding n^{th} coins and including n^{th} coins>

- Eexcluding:
  - dp[n-1][sum]
- Including:
  - 1 + dp[n] [sum – coins[n]]

min (excluding, including)

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	…	…			Sum = S
0 coin								
1 coins								
2 coins								
…								
…								
n coins								

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	…	…			Sum = S
0 coin	0							
1 coins	0							
2 coins	0							
…	0							
…	0							
	0							
	0							
n coins	0							

Coin Change (Minimum number of coins)

	Sum = 0	Sum = 1	Sum = 2	…	…			Sum = S
0 coin	0	∞	∞	∞	∞	∞	∞	∞
1 coins	0							
2 coins	0							
…	0							
…	0							
	0							
	0							
n coins	0							

Bottom up implementation 1

```
for (i = 0; i <= n ; i++) {
        for (j = 0; j <= sum; j++) {
            if(j==0) dp[i][j] = 0; //Can be initialized outside loop
            else if(i==0) dp[i][j] = INF; //Can be initialized outside loop

            else if(j < coins[i - 1])
                        dp[i][j] =  dp[i-1][j];
            else
                        dp[i][j] = min( dp[i-1][j], 1 +dp[i] [ j – coins[i - 1] ] );
        }
    }
```

Knapsack Problem [0-1]

- Given weights **and** values of N items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

Knapsack Problem [0-1]

- Input: N = 3, W = 4
- values[] = {1,2,3}
- weight[] = {4,5,1}
- Output: 3

- Input: N = 3, W = 3
- values[] = {1,2,3}
- weight[] = {4,5,6}
- Output: 0

Recurrence Relation

- Smaller sub problems and the relation between them ?

Recurrence Relation

- Smaller sub problems and the relation between them ?
  - dp[i][w] = max(val[i] + dp[i - 1][w - wt[i]], dp[i - 1][w]);

Knapsack Problem [0-1]

	W = 0	W = 1	W = 2	…	…			W = S
0 obj								
1 obj								
2 obj								
…								
…								
n obj								

Knapsack Problem [0-1]

	w = 0	w = 1	w = 2	…	…			w = W
0 obj	0	0	0	0	0	0	0	0
1 obj	0							
2 obj	0							
…	0							
…	0							
	0							
	0							
n obj	0							

Knapsack Problem Variations

- Each object has unlimited supply
- You may take a fraction of a object